

Homework Week 7 [30 points + 12 extra credit points]

Due: March 4, 2026 11:59pm PT.

Upload a zip archive of your project on <https://brightspace.usc.edu> .

Your project must consist of a folder `hw7_<first_name>_<last_name>/` and python files (python modules) and data file(s) inside inside:

- `hw7_<first_name>_<last_name>/vehicle.py` # place your functions from task 1 here
- `hw7_<first_name>_<last_name>/complex_numbers.py` # place your functions from tasks 2 here
- `hw7_<first_name>_<last_name>/shapes.py` # place your functions from tasks 3 here
- `hw7_<first_name>_<last_name>/expressions.py` # place your functions from tasks 4 here
- `hw7_<first_name>_<last_name>/tests.py` # place your tests here, see example in HW4

Use of generative AI:

- is not allowed for implementation or code completion in this homework, **Specifically for regular expressions (!!!) - do not use AI assistance. Also if you use functions/regex patterns other than what was explained in class slides, we will consider this as a violation of AI usage requirements.** If you really need to use something in regular expressions that wasn't explained in class ask on Piazza first.
- is allowed for concept clarification (e.g. "What is function signature in python?", "How can I check if a character is in a Python string?"), you must include the name of the tool you used (e.g. Copilot, ChatGPT, Gemini, etc.) and the list of questions/prompts you asked in a separate prompts.txt,

Task 1. OOP - Vehicle Inheritance [10 points]

Create a Python class Vehicle that has three instance attributes:

- make (the manufacturer of the vehicle)
- model (the specific model of the vehicle)
- year (the year the vehicle was made)

It should also include a method `__str__` that returns a string containing the year, model and make of the vehicle.

Then, create two subclasses of Vehicle: Bike and Car.

Bike Class:

DSCI 510 – Spring 2026 – University of Southern California

Instructor: Dr. Alexey Tregubov

- The Bike class should have an additional instance attribute `bike_type` (a string that indicates whether the bike is "road" or "mountain" type) and also override the `__str__` method to include the `bike_type` of bike.

Car Class:

- The Car class should have an additional instance attribute `is_electric` (a boolean indicating if the car is electric) and override the `__str__` method to include information on whether the car is electric.
- Implement a method `driving_range(self, energy)` that calculates the driving range based on the car's energy source.
- If the car is electric, `energy` represents battery percentage (0-100), and the method calculates the range based on `miles_per_battery_percent * energy`.
- For non-electric cars, `energy` represents gallons of fuel, and the calculation is based on `miles_per_gallon * energy`.

For calculating the range for the car:

- `miles_per_battery_percent = 4`
- `miles_per_gallon = 30`

Make these two parameters instance-level attributes and add them into your `__init__()` method with default values above: `__init__(..., miles_per_battery_percent = 4, miles_per_gallon = 30)`.

Example:

```
my_car = Car("Tesla", "Model S", 2020, is_electric=True)
print(my_car)
# Expected output: "2020 Tesla Model S, Electric"
my_non_electric_car = Car("Ford", "Mustang", 1965)
print(my_non_electric_car)
# Expected output: "1965 Ford Mustang, Non-Electric"
# Demonstrating the driving range method:
print("Electric Car Driving Range:", my_car.driving_range(50)) # 50%
battery
```

```
# Expected output: "Electric Car Driving Range: 200"
print("Non-Electric Car Driving Range:",
my_non_electric_car.driving_range(10)) # 10 gallons
# Expected output: "Non-Electric Car Driving Range: 300"
my_bike = Bike("Trek", "Domane", 2019, "road")
print(my_bike)
```

DSCI 510 – Spring 2026 – University of Southern California

Instructor: Dr. Alexey Tregubov

Expected output: "2019 Trek Domane, road type"

Task 2. OOP - Arithmetic Overloading [10 points + 2 extra credit points]

Implement a Python class `ComplexNumber` that represents a complex number with real and imaginary parts. The class should override the `+` (addition) and `*` (multiplication) operators to allow adding and multiplying two `ComplexNumber` instances, respectively. (More information on Complex numbers is present at the end of this document)

The class should have an `__init__` method to set the real and imaginary parts, an `__add__` method to handle addition, and an `__mul__` method to handle multiplication. Both these methods should return a complex number with the respective operations performed. It should also have a `__str__` method which prints the output in the format shown below.

Example:

```
a = ComplexNumber(2, 3)
```

```
b = ComplexNumber(1, 7)
```

```
c = a + b # Should perform complex addition
```

```
d = a * b # Should perform complex multiplication
```

```
print(c) # Output: "ComplexNumber with real part 3 and imaginary part 10"
```

```
print(d) # Output: "ComplexNumber with real part -19 and imaginary part 17"
```

(2points) Extra credit: if you make `__add__` and `__mul__` operations work for any number. For example:

```
a = ComplexNumber(2, 3)
```

```
b = 5
```

```
c = a + b # Should perform complex addition
```

```
print(c) # Output: "ComplexNumber with real part 7 and imaginary part 3"
```

More info about Complex Numbers

Complex numbers are numbers that consist of a real part and an imaginary part. They are typically written in the form $a + bi$, where a is the real part, b is the imaginary part, and i is the square root of -1 .

Addition of Complex Numbers

The addition of two complex numbers is performed by adding their real parts together and their imaginary parts together separately. If you have two complex numbers $a + bi$ and $c + di$, their sum is given by:

$$(a + bi) + (c + di) = (a + c) + (b + d)i$$

Multiplication of Complex Numbers

The multiplication of two complex numbers involves distributing the terms and using the fact that $i^2 = -1$. Given two complex numbers $a + bi$ and $c + di$, their product is calculated as follows:

$$(a + bi)(c + di) = ac + adi + bci + bdi^2$$

Since $i^2 = -1$, we can simplify the expression to:

$$\begin{aligned} &= ac + adi + bci - bd \\ &= (ac - bd) + (ad + bc)i \end{aligned}$$

Example

Given the complex numbers $2 + 3i$ and $1 + 7i$:

- **Addition:** $(2 + 3i) + (1 + 7i) = (2 + 1) + (3 + 7)i = 3 + 10i$
- **Multiplication:** $(2 + 3i)(1 + 7i) = (2 * 1 - 3 * 7) + (2 * 7 + 3 * 1)i = -19 + 17i$

Task 3. OOP - Shape Management System [10 points]

Create a Python class `Shape` with a method `area()` that returns the area of the shape. Then, create two subclasses, `Rectangle` and `Circle`, that override the `area()` method. For `Rectangle`, the area should be calculated as `length * width`; for `Circle`, the area should be calculated as $\pi * \text{radius}^2$.

DSCI 510 – Spring 2026 – University of Southern California

Instructor: Dr. Alexey Tregubov

The method `area()` should initially raise a `NotImplementedError` with the message "This method should be overridden by subclasses", indicating that any subclass of `Shape` must provide its own implementation of the `area()` method.

Additionally, implement method overloading for the `Rectangle` class to allow creating an instance either by providing both length and width or by providing just a single side parameter to create a square.

In addition to the `area()` method, both the `Rectangle` and `Circle` subclasses should implement a new method named `dimensions()` that returns the dimensions of the shape in a dictionary format. For the `Rectangle`, this method should return a dictionary containing either the length and width for rectangles or a single side for squares. For the `Circle`, the `dimensions()` method should return a dictionary with a single key `radius` that maps to the circle's radius.

All the attributes such as length, width, side and radius will be given as integers.

Example:

```
rectangle = Rectangle(length=5, width=10)
print(rectangle.area()) # Expected Output: 50
print(rectangle.dimensions()) # Expected Output: {'length': 5, 'width': 10}
square = Rectangle(side=5)
print(square.area()) # Expected Output: 25
print(square.dimensions()) # Expected Output: {'side': 5}
circle = Circle(radius=7)
print(circle.area()) # Expected Output: 153.94 (Rounded off to two digits
# after decimal)
print(circle.dimensions()) # Expected Output: {'radius': 7}
```

DSCI 510 – Spring 2026 – University of Southern California

Instructor: Dr. Alexey Tregubov

Task 4. (Extra credit) Regular expression practice [10 points, extra credit]

Implement the following functions, **only used regular expression functions/patterns explained in class (see lecture slides)**:

4.1 Parse a string and return a list of **all 3-digit numbers**.

Function: `def extract_three_digit_numbers(text: str) -> int:`

Example Usage:

```
text = "The codes are 101, 42, 999, and the year is 2024."
```

```
print(f"Input: '{text}')
```

```
print(f"Output: {extract_three_digit_numbers(text)}")
```

```
# Expected Output: ['101', '999', '202']
```

4.2 Find and extract the full URL, but *only* if it starts with `http://` or `https://`.

Function: `def extract_secure_urls(text: str) -> str:`

Example Usage:

```
text = "Visit our site: https://www.example.com/page-1 or the old one: http://legacy.org/index.html. Local link: file://local/path"
```

```
print(f"Input: '{text}')
```

```
print(f"Output: {extract_secure_urls(text)}")
```

```
# Expected Output: ['https://www.example.com/page-1', 'http://legacy.org/index.html']
```

4.3 Strip all simple HTML tags (e.g., `<b>`, `<i>`, `</p>`) from a string.

Function: `def strip_html_tags(text: str) -> str:`

Example Usage:

```
text = "This is a <b>bold</b> word and an <i>italic</i> phrase, ending with a <p>paragraph tag</p>."
```

```
print(f"Input: '{text}')
```

```
print(f"Output: {strip_html_tags(text)}")
```

```
# Expected Output: "This is a bold word and an italic phrase, ending with a paragraph tag."
```

4.4 Find dates in the format `MM-DD-YYYY` and reformat them to `YYYY/MM/DD..`

Function: `def reformat_date(text: str) -> str:`

Example Usage:

```
text = "The event is scheduled for 10-25-2025, but the deadline passed on 01-15-2024."
```

```
print(f"Input: '{text}')
```

```
print(f"Output: {reformat_date(text4)}")
```

DSCI 510 – Spring 2026 – University of Southern California

Instructor: Dr. Alexey Tregubov

Expected Output: "The event is scheduled for 2025/10/25, but the deadline passed on 2024/01/15."